School of Tech
part of accenture

Generation

# Databases

# A front-to-back system

Databases like Postgres are the "backend" of most systems - they provide the persistent storage (and can also do various calculations for us).

We have been using Adminer as a "front end" on our data (especially when running Select statements).

Typically, we will write that front-end ourselves - which means connecting to the database from our Python code.

# Python code and databases

We will need our python code to:

- Make a connection to the database
- Query the data from it with a `SELECT` statement (or, later, `INSERT`, or `UPDATE`)
- Present the results to us on-screen

# Python code and databases

> What data might the python code need in order to perform those steps?

- Address of the server
- Port of the server
- Name of the database
- Name of the table
- Query parameters (for WHERE, ORDER BY, etc)

These are usually passed in as environment variables.

# SQL in Python

The library `psycopg` is a modern implementation of a PostgreSQL client library for Python - we can use it to connect and run our SQL, i.e. allow us to interact with a database purely through Python.

# Sample requirements.txt

A sample is pre-prepared for us.

> Let's look inside `handouts/requirements.txt`

This contains a sample set of useful libraries we will use.

# Exercise - requirements

> Everyone do `Part 4.1 - requirements` now

- Open a terminal in folder `handouts`
- Activate a new virtual env
- Install the dependencies from `requirements.txt` with:
  - `python3 -m pip install -r requirements.txt` (MacOS / Unix / GitBash)
  - `python -m pip install -r requirements.txt` (Windows)

# Using env vars

We do not usually want sensitive information like connection details or passwords in our python files!

A special file called `.env` can be made with dummy values for our testing, which will have real values in production systems.

In production systems, these values can be created without them ever being saved in our code.

# Using env vars

A standard way to write our code is use the `dotenv` utility to load them from the `.env` file. They can then be accessed in our application like so:

```
from dotenv import load_dotenv

# Load environment variables from .env file
load_dotenv()
host_name = os.environ.get("POSTGRES_HOST")
database_name = os.environ.get("POSTGRES_DB")
user_name = os.environ.get("POSTGRES_USER")
user_password = os.environ.get("POSTGRES_PASSWORD")
```

# Making a connection

The `psycopg` utility has a `connect()` function for creating a connection :

```
connection = psycopg.connect(f"""
    host={host_name}
    dbname={database_name}
    user={user_name}
    password={user_password}
    """)
```

(This code assumes we have the variables `host_name`, `database_name`, `user_name`, `user_password` from the previous slide)

# Making a connection "properly"

We should use a *context* for the connection, like so:

```
try:
    with psycopg.connect(f"""
        host={host_name}
        dbname={database_name}
        user={user_name}
        password={user_password}
        """) as connection:

        # More code here
        ...

        # The connection will automatically close here
except Exception as ex:
    print('Failed to:', ex)
```

# Opening a cursor

A cursor is an object that lets us manipulate the database, in the context of a database operation:

The function to create one is on the `connection` object, like so:

```
cursor = connection.cursor()
```

We can use this to run queries (SQL) on the database.

# An insert query

For inserting data, we give the `execute()` function some SQL with placeholders in it, and a tuple of all the data, like so:

```
sql = """
    INSERT INTO person (first_name, last_name)
    VALUES (%s, %s)
    RETURNING person_id, first_name, last_name
"""
data_values = ('first_name', 'last_name')

cursor.execute(sql, data_values)
```

The `execute()` function will do string substitutions for us in order - but we need to match the correct number of columns in the INSERT clause, VALUES clause, and tuple.

# INSERT with RETURNING

We can get back the id that the server created with the `RETURNING` keyword.

This is especially useful when writing code (like python) that is adding the data for us.

# Committing the data

"Committing" in database terms means "save any updates".

Adminer has been doing this for us - in code we need to do this ourselves, like so:

```
connection.commit()
```

# Selecting data

To select data we also use a cursor. This will return all the matching data to us:

```
cursor.execute('SELECT column_1, column_2 FROM table_name')
rows = cursor.fetchall()
```

We can also use any other valid clauses like `WHERE`, `ORDER BY`, and SQL functions.

# Select results

The result we get (`rows` here) will be an array with an entry per row.

Each row will also be an array, of the column values, e.g:

```
rows = [
    ['row1_col1_value', 'row1_col2_value'],
    ['row2_col1_value', 'row2_col2_value']
]
```

...and so on. The data type in the array will depend on that of the columns (i.e. strings, numbers, booleans).

# Tidying up

When we are done with a cursor or connection, we need to close each of them to release the resources they are holding onto:

```
# Closes the cursor so will be unusable from this point
cursor.close()
```

We only need to manually close the connection if we didn't use a context:

```
# Closes the connection to the DB, ALWAYS do this
connection.close()
```

# Exercise prep - standard table

> The instructor will give you the sql in file `solutions/person_database.sql`

- This has a database called `test`
- And inside that, a table called `person`
- It matches the values in the `handouts/.env` file

**If** you **don't** have a table matching this, please run all this sql in Adminer.

# Exercise prep - skeleton file

> The instructor will show you the file

- A template file `handouts/my_db_app.py` is provided
  - It already has the imports
  - It already loads the environment variables from an `.env` file
- The file doesn't do anything... yet!
  - It won't compile until it is filled in

# Exercise

> Complete the steps described in `Part 4.2 - add more code` in `database-exercises.md`

- Check your `.env` file has values matching your database name and table name
  - If not, update it to match or run the sample sql
- Edit the `my_db_app.py` file
  - Add the missing code as per the `TODO` sections
- Run your file to test it!

# Discussion

> How did you get on connecting to the database, querying it, and showing the data?

# Example of using context

The psycopg docs have a nice example of using a context for a connection and another for a cursor.

- https://www.psycopg.org/docs/connection.html

...This is the sort of thing one does outside of this exercise :-)

# Terms and Definitions - recap

**Database**: An organised collection of data, generally stored and accessed electronically from a computer system.

**Relational Database**: A digital database based on the relational model of data.

**SQL**: A domain-specific language used in programming and designed for managing data held in a relational database management system.

**Query**: A precise request for information retrieval with database and information system.

# Terms and Definitions - recap

**DDL**: A data description language is a syntax for creating and modifying database objects such as tables, indexes, and users.

**DML**: A data manipulation language is a computer programming language used for inserting, deleting, and updating data in a database.

**Primary Key**: A specific choice of a minimal set of attributes (columns) that uniquely specify a tuple (row) in a relation (table).

**Foreign Key**: A set of attributes in a table that refers to the primary key of another table.

# Overview - recap

- What is a database?
- SQL
- DDL / DML
- Modelling
- Connecting to a database from Python

# Learning Objectives - recap

- Explain what a database is and how to utilise one
- Describe SQL and it's categories, DDL and DML
- Identify modelling relationships in data
- Setup your own database and interact with it using Python

# Further Reading

- Learn SQL with Code Academy
- Database Design
- psycopg
- Database definitions
- Database definitions discussion

# Emoji Check:

On a high level, do you think you understand the main concepts of this session? Say so if not!

1. 😢 Haven't a clue, please help!
2. 😕 I'm starting to get it but need to go over some of it please
3. 😐 Ok. With a bit of help and practice, yes
4. 🙂 Yes, with team collaboration could try it
5. 😀 Yes, enough to start working on it collaboratively